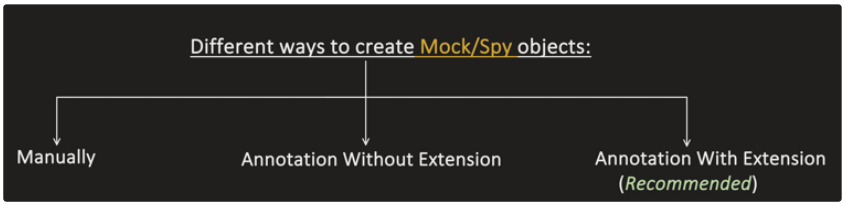
@Mock, @InjectMocks (Junit & Mockito Integration)



Pre-requisite:

ncept && Coding

1. **Manually:**

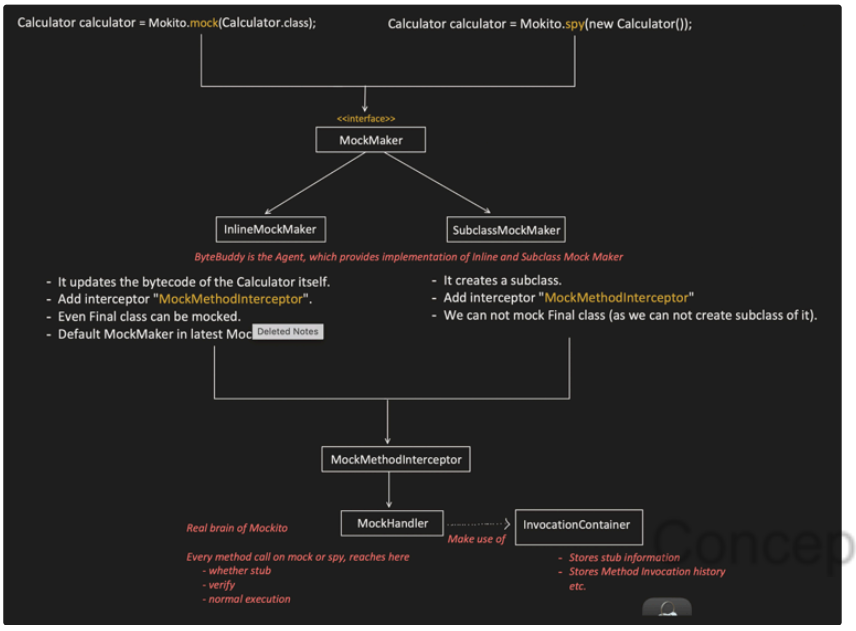
Mock:

```
@Test
void placeOrderSuccess() {

    Calculator calculator = Mockito.mock(Calculator .class);

    when(calculator.multiply(4, 2)).thenReturn(10);

    int output = calculator.multiply(4, 2);

    assertTrue(result);

    verify(paymentService).pay(10);
 }

```

Spy:

```java
@Test
void placeOrderSuccess() {

    Calculator calculator = Mockito.spy(new Calculator());

    Mockito.doReturn(10).when(calculator).multiply(4, 2);

    int product= calculator.multiply(4, 2); //mocked
    assertEquals(10, product);

    int sum = calculator.add(4, 2); //call real method

 }

```

In Architecture topic, we already seen what internally happens when we do:



Concept && Coding

These classes are present in "Mockito core" library.

So, we are using the Mockito manually:

- Creating the mock/spy manually.
- Stubbing manually
- Verifying manually

all we need is **"mockito-core"** library:

```pom.dependency
    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
        <scope>test</scope>
        </dependency>


```

<u>Junit Lifecycle and Manual Mock/Spy object creation:</u>

1. In **@BeforeEach** method:

```
@BeforeEach
void setup() {
      calculator = Mockito.mock(Calculator.class);
}

```

Safe, because:

- Fresh mock per test.
- No need to reset.
- No need to clean up any resource.

2. In **@BeforeAll method**:

```
@BeforeAll
void setup() {
      calculator = Mockito.mock(Calculator.class);
}

```

Not safe, because:

- Same mock shared across tests
- stubbing leak into other tests

Concept && Coding

3. In **Field Initializer**:

PER_METHOD lifecycle:

```
public class CalculatorTest {

  Calculator calculator = Mockito.mock(Calculator.class);

    @BeforeEach
    void setup() {
          . . .
    }

}

```

Safe, because:

- New test class instance is created per test method.
- So fresh fields, fresh mocks

PER_CLASS lifecycle:

```
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class CalculatorTest {

  Calculator calculator = Mockito.mock(Calculator.class);

   @Test
    void test1() {
         calculator.multiply(10, 2);
        Mockito.verify(calculator).multiply(10,2);
    }

    @Test
    void test2() {
```

```
        calculator.pay(5, 6);
      /*
      Fails because, mock object remembers the interaction
      from previous test method too.
      So expectation is 1 call to multiply method
       but actually its 2.
        */
        Mockito.verify(calculator).multiply(5, 6); //FAILS
    }

}

```

Not Safe, because:

- Test class instance is created only once and used by all test methods.
- So same mock object is used by all test methods and thus:
- Risk of leaking Stubs
- Interaction history (verify)

Etc.

2. **Annotation Without Extension:**

```
public class OrderService {

    private final PaymentGateway gateway;
    private final OrderRepository repo;

    public OrderService(PaymentGateway gateway, OrderRepository repo)
{
        this.gateway = gateway;
        this.repo = repo;
    }

    public boolean placeOrder(int amount) {

                ……….do something……..
    }

}

```

Concept && Coding

PaymentGateway.java

```

public interface PaymentGateway {
      boolean charge(int amount);
}
```

OrderRepository.java

```
public interface OrderRepository {
    void save();
}
```

```
class OrderServiceTest {

    AutoCloseable closable;
      /*
      At this moment, nothing happens.
      only
      @Mock metadata is added with this field.
      gateway = null
      repo = null
    */

    @Mock
```

```java
    PaymentGateway gateway;
    /*
      At this moment, nothing happens.
        only
      @Mock metadata is added with this field.
      gateway = null
      repo = null
      */
    @Mock
    OrderRepository repo;
    /*
    At this moment, nothing happens.
    only
    @InjectMock metadata is added with this field.
    service = null
    */
    @InjectMocks
     OrderService service;

    @BeforeEach
    void init() {
      /*
      *** All major works happen in this:

  Step1: Reflection scan:
    - Find fields with @Mock annotation.
    - Find fields with @Spy annotaiton.
    - Find fields with @InjectMocks annotation.

  Step2: Create Mock object for @Mock fields
    - Internally it will do:
    gateway = Mockito.mock(PaymentGateway.class);
    repo = Mockito.mock(OrderRepository.class);

  Step3: InjectMocks handling
    - Create real object of the field annotated with @InjectMocks.
    - And in real object member variables, set the mock objects, which are created in Step2.

    service = new OrderService(gateway, repo);


  Step4: Stores internal state information
    - Fields which are mocked.
    - Fields which are Injected.

  This is stored in ThreadLocal, and even after test method is finished,
  this data is still present for the running Thread.

    so, problem is, if we have a thread Pool, and resource clean up is not happen.
  Then Same thread might pick different test and might see un-predictable behavior.


  Also, if resource clean up is not happens, it might consume lot of space
  unnecessarily.
        */
        closable = MockitoAnnotations.openMocks(this);
    }

    @AfterEach
    void cleanup() throws Exception {
      /*
      After each test, we need to release the resources,
      so that GC can clean it up.
        */
         closable.close();
    }

    @Test
    void placeOrder_success() {
        Mockito.when(gateway.charge(100)).thenReturn(true);

        boolean result = service.placeOrder(100);

        assertTrue(result);
        Mockito.verify(repo).save();
    }
}

```

Concept && Coding

These Mockito Annotations classes are present in "Mockito core" library.

So, all we need is "mockito-core" library:

```
<dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <scope>test</scope>
</dependency>

```

<u>Cons of "Annotations without Extension":</u>

- **Boilerplate: In every class, we have to do:**

AutoCloseable closable;

@BeforeEach

closable= openMocks(this);

@AfterEach

closable.close();

- **Easy to forget close()**

Concept && Coding

- Manually need to wire the mock creation with JUnit lifecycle.
  Need to make sure that its added it @BeforeEach and closed in @AfterEach.
  What if by mistake its added in @BeforeAll.

  So, manually need to do the wiring in JUnit lifecycle.

<u>Before we proceed to "Annotation with Extension"</u>

in Step3 I mentioned that "in real object member variables mock object is set"

<u>Step3:</u> InjectMocks handling

- Create real object of the field annotated with @InjectMocks.
- And in real object member variables, set the mock objects, which are created in Step2.

service = new OrderService(gateway, repo);

So, one question might come is: **<u>What are the different ways to set the member variables</u>?** And what are the Rules?

1. Constructor Injection (First Priority)

2. Setter Injection (Second Priority)
3. Field Injection (Last Priority)

Lets try to cover all possible scenarios:

Constructor Injection:

Rules:

- Always pick the constructor with most parameter.
- Do not guess value for primitive data types.
- If Mock is not available for any constructor parameter object, use null (but do not create mock for that object).
- If not able to fill any parameters of picked constructor fallback to default constructor.

Use case #1: Only 1 Constructor present: **Exact Match**

```java
public class OrderService {

    private final PaymentGateway gateway;
    private final OrderRepository repo;

    public OrderService(PaymentGateway gateway, OrderRepository repo)
{
        this.gateway = gateway;
        this.repo = repo;
    }

    public boolean placeOrder(int amount) {

              ……….do something……..
    }

}

```

Concept && Coding

```java
class OrderServiceTest {

    @Mock
    PaymentGateway gateway;

    @Mock
    OrderRepository repo;

    @InjectMocks
    OrderService service;
.
    .
    .

}

```

Internally it will do:

→ service = new OrderService(gateway, repo)

→ Found the Constructor which has all the Mock object Types.

Use case #2: More than 1 Constructor present: **able to resolve the dependencies**

```java
public class OrderService {

    private final PaymentGateway gateway;
    private final OrderRepository repo;
    private final SalesRepository salesRepo;

    public OrderService(PaymentGateway gateway, OrderRepository repo)
{
```

```
        this.gateway = gateway;
        this.repo = repo;
        this.salesRepo = null;
    }

    public OrderService(PaymentGateway gateway, OrderRepository repo, SalesRepository salesRepo) {
        this.gateway = gateway;
        this.repo = repo;
        this.salesRepo = salesRepo;
    }


    public boolean placeOrder(int amount) {

                ……….do something……..
    }

}

```

```
class OrderServiceTest {

    @Mock
    PaymentGateway gateway;

    @Mock
    OrderRepository repo;

    @InjectMocks
    OrderService service;
.
    .
    .

}

```

Concept && Coding

Always pick the constructor with most parameter :

Constructor picked :

OrderService(PaymentGateway gateway, OrderRepository repo, SalesRepository salesRepo)

PaymentGateway gateway = Mock is available

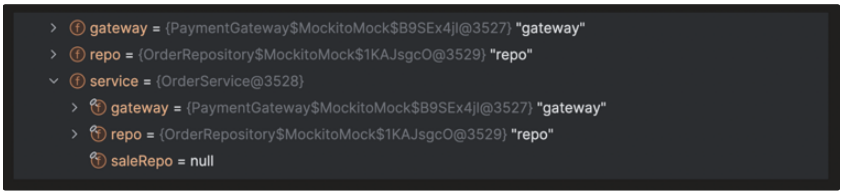
OrderRepository repo = Mock is available

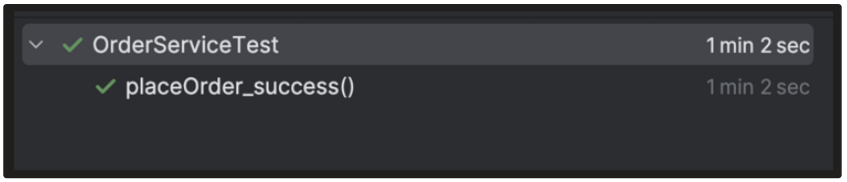
If Mock is not available for any constructor parameter object, use null (but do not create mock for that object).

SalesRepository salesRepo = No Mock object is available, so it will pass NULL.

@InjectMock able to create OrderService object:

service = new OrderService(gateway, repo, null)





Use case #3: **More than 1 Constructor present: NOT able to resolve the dependencies**

```java
public class OrderService {

    private final PaymentGateway gateway;
    private final OrderRepository repo;
    int var;

    public OrderService(PaymentGateway gateway, OrderRepository repo) {
        this.gateway = gateway;
        this.repo = repo;
    }

    public OrderService(PaymentGateway gateway, OrderRepository repo, int var) {
        this.gateway = gateway;
        this.repo = repo;
        this.var= var;
    }


    public boolean placeOrder(int amount) {

                ……….do something……..
    }

}

```

```java

class OrderServiceTest {

    @Mock
    PaymentGateway gateway;

    @Mock
    OrderRepository repo;

    @InjectMocks
    OrderService service;
.
    .
    .

}

```

Concept && Coding

Always pick the constructor with most parameter :

Constructor picked :

OrderService(PaymentGateway gateway, OrderRepository repo, int var)

PaymentGateway gateway = Mock is available

OrderRepository repo = Mock is available

Do not guess value for primitive data types.

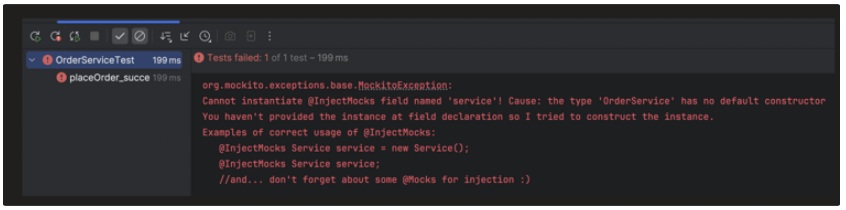
int var , it is not able to resolved the parameter of the constructor picked.

If not able to fill the parameters of picked constructor fallback to default constructor.

Try to invoke:

service = new OrderService();

But we don't have default constructor in OrderService class, so it failed.



<u>Setter Injection:</u>

<u>Rules</u>:

- Always have default no-arg constructor present.
  why? because 1st priority is Constructor Injection but to move to Setter Injection, it should fail to resolve the parameters dependency of picked constructor and fallback to default constructor and then call setter methods on the newly object created.

- Setter method, must follow naming convention.

setSalesRepo(SalesRepository salesRepo)

Concept && Coding

not:

salesRepo(SalesRepository salesRepo)

addSalesRepo(SalesRepository salesRepo) etc....

- Setter method must exactly take 1 argument

setAll(A a, B b) // ignored

- Setter injection do not work for final fields.

- If Setter Injection is not possible for certain fields, then for those fields it try to do FIELD INJECTION through reflection (even private fields will get set but not final)

```
public class OrderService {

    private PaymentGateway gateway;
    private OrderRepository repo;
    private SalesRepository salesRepo;

       public OrderService() {  }

    public OrderService(PaymentGateway gateway, OrderRepository repo,
int var) {
```

```
            this.gateway = gateway;
            this.repo = repo;
        }

      public void setGateway(PaymentGateway gateway) {
             this.gateway = gateway;
       }

       public boolean placeOrder(int amount) {
               ……….do something……….
       }

}

```

```

class OrderServiceTest {

    @Mock
    PaymentGateway gate;

    @Mock
    OrderRepository repo;

    @Mock
    SalesRepository salesRepo;

    @InjectMocks
    OrderService service;
.
    .
    .

}

```

Step1: Go for constructor Injection:

Concept && Coding

Pick the constructor is most parameters:

Constructor picked:

OrderService(PaymentGateway gateway, OrderRepository repo, int var)

But

It did not able to satisfy all the parameter of picked constructor *(int var it do not able to guess the value)*

So,

It fall back to Default no-arg constructor and able to find it.

OrderService service = new OrderService();

Once object is created, now first Setter Injection → then field injection is tried to initialize all the fields.

Step2: Go for Setter Injection

OrderService object has 3 fields :

private PaymentGateway gateway;

private OrderRepository repo;

private SalesRepository salesRepo;

It picks first field:

private PaymentGateway gateway;

Then it will try to look for its setter method:

setGateway(PaymentGateway gateway)

Setter method found:

Then it will check, do I have Mock object for this field type "PaymentGateway"

@Mock

PaymentGateway gate;

Yes it has, so it will invokes this ***setGateway*** method.

Now in OrderService object

Concept && Coding

- gateway = Mock Gateway is set.

Now ,

OrderService object has 2 more fields which need to be set:

private OrderRepository repo;

private SalesRepository salesRepo;

And for both the fields, it did not able to find the setter method.

so,

<u>Step3: Go for field Injection for these fields</u>

private OrderRepository repo = @Mock OrderRepository repo;

private SalesRepository salesRepo = @Mock SalesRepository salesRepo;

Even private fields are set.

1. **Annotation With Extension:**

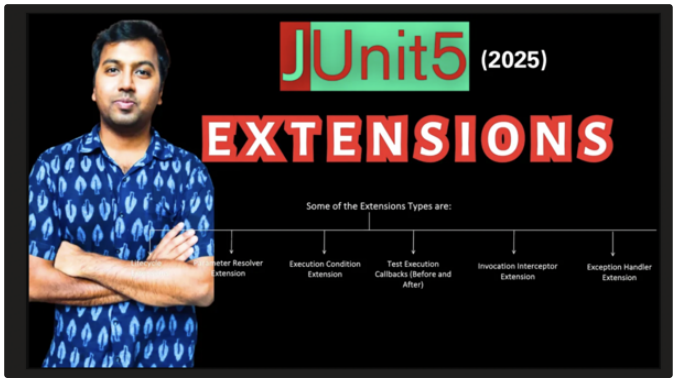
Its similar to "Annotation Without Extension"

Only thing is:

- We don't have to worry about openMock()
- Closing the resource
- Wiring into the Junit lifecycle.

Its all managed by Extension.

If any doubts with extension, kindly check the in-depth video on it in Junit5 series:



Concept && Coding

```java
public class OrderService {

    private final PaymentGateway gateway;
    private final OrderRepository repo;

    public OrderService(PaymentGateway gateway, OrderRepository repo) {
        this.gateway = gateway;
        this.repo = repo;
    }

    public boolean placeOrder(int amount) {

            ..........do something........
    }

}

```

PaymentGateway.java

```java
public interface PaymentGateway {
      boolean charge(int amount);
}

```

OrderRepository.java

```java
public interface OrderRepository {
    void save();
}

```

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    PaymentGateway gateway;

    @Mock
    OrderRepository repo;

    @InjectMocks
     OrderService service;

    @Test
    void placeOrder_success() {
        Mockito.when(gateway.charge(100)).thenReturn(true);
        boolean result = service.placeOrder(100);
        assertTrue(result);
        Mockito.verify(repo).save();
    }
}

```

1. ***MockitoExtension*** *enable hooks in JUPITER code.*

Gets Lifecycle callbacks like:

- Before each Test
- After each test

```java
public class MockitoExtension implements BeforeEachCallback, AfterEachCallback, ParameterResolver {
```

And this **MockitoExtension** comes in library:

Concept && Coding

```xml
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.11.0</version>
    <scope>test</scope>
</dependency>


```

And this ***mockito-junit-jupiter*** also fetches "***mockito-core***" library, so we don't specifically need to add ***mockito-core*** library:

2. Before each Test method, it internally calls similar to:

    closable = **MockitoAnnotations.openMocks(this);**

@Override

public void beforeEach(final ExtensionContext context) {

. . .

```
MockitoSession session =
Mockito.mockitoSession()
.initMocks(testInstances.toArray())
.strictness(actualStrictness)
.logger(new MockitoSessionLoggerAdapter(Plugins.getMockitoLogger()))
.startMocking();

. . .

}
```

3. After each Test method, it internally calls similar to:

   closable.**close();**

```
@Override
public void afterEach(ExtensionContext context) {
...
```

Concept && Coding

```
context.getStore(MOCKITO)
.remove(SESSION, MockitoSession.class)
.finishMocking(context.getExecutionException().orElse(null));

...
}
```

Construction, Setter and Field injection rules and process is same like Annotation without Extension.